

Exercices chapitre 4-Structures de données

Exercice 4.1

Travail sur les Piles

1) En utilisant le module `Stack` écrire les fonctions suivantes:

a) `affiche_pile pile` de type `int Stack.t -> unit` qui affiche chacun des éléments de la pile `pile`, en commençant par le haut de la pile. Que devient la pile donnée en argument après exécution de cette fonction?

b) `liste_to_pile liste pile` de type `'a list -> 'a Stack.t -> unit` qui transfère les éléments de `liste` dans la pile `pile`.

c) `pile_to_liste pile` de type `'a Stack.t -> 'b list` qui renvoie la liste des éléments contenus dans la pile `pile`.

d) `iter_pile f p` de type `('a -> 'unit) -> 'a Stack.t -> unit` qui applique successivement la procédure `f` à tous les éléments de `pile`.

2) A l'aide de `iter_pile` écrire une fonction d'affichage de tous les éléments d'une pile, puis une fonction de calcul de la somme des éléments d'une pile.

Exercice 4.2

Piles et permutations

On dit qu'une permutation $(a_1, a_2, \dots, a_n)$ de $(1, \dots, n)$ peut être engendrée par une pile lorsqu'il est possible, à partir de la séquence d'entrée $(1, 2, \dots, n)$ et d'une pile (initialement vide), de produire la séquence de sortie $(a_1, a_2, \dots, a_n)$ en utilisant les opérations suivantes :

- empiler l'élément suivant de la séquence d'entrée ;
- ou dépiler le sommet de la pile et l'imprimer à l'écran.

Par exemple, si `E` et `D` désignent respectivement chacune des deux opérations permises, la permutation $(2, 3, 1)$ est engendrée par la suite d'opérations : `EEDEDD`. On représentera en CAML une permutation par le type `int list`.

1) Parmi les permutations suivantes, lesquelles peuvent être engendrées par une pile ? $(3, 1, 2)$, $(3, 4, 2, 1)$, $(4, 5, 3, 7, 2, 1, 6)$, $(3, 5, 7, 6, 8, 4, 9, 2, 10, 1)$.

2) Écrire une fonction Caml déterminant si une permutation peut être engendrée par une pile. Dans le cas d'une réponse positive, la fonction affichera la suite d'opérations permettant de la produire. La fonction sera de signature `engendrable : int list -> bool` et par exemple, `engendrable [2;3;1]` affichera `EEDEDD` et renverra le booléen `true`.

Exercice 4.3

Parcours de graphes non orientés

On utilisera librement dans cet exercice les fonctions des modules `Stack` et `Queue`.

On appelle dans cet exercice graphe (non orienté) un couple $G = (S, A)$ de deux ensembles:

S , l'ensemble des sommets de G , que l'on prendra ici de la forme $\llbracket 0, n-1 \rrbracket$

et A l'ensemble des arêtes de G . C'est un ensemble de paires de la forme $\{i, j\}$ telles que $(i, j) \in S^2$ et $i \neq j$.

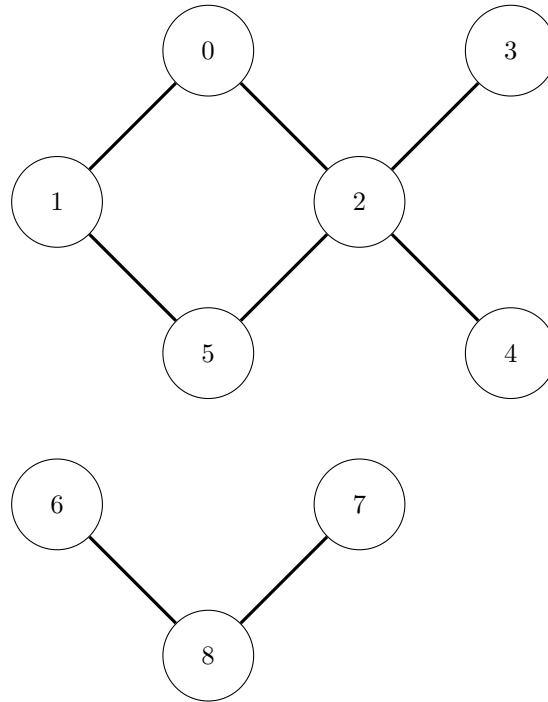
Si i et j sont deux sommets de G , alors on appelle chemin c de longueur $p \in \mathbb{N}$ de i vers j une suite finie $c = (s_0, s_1, \dots, s_p)$ de sommets de G tels que pour tout $i \in \llbracket 0, p-1 \rrbracket$, $\{s_i, s_{i+1}\} \in A$.

On appelle composante connexe du sommet $i \in S$, l'ensemble des sommets $j \in S$ tels qu'il existe un chemin de longueur p de i vers j . Par convention, le sommet i est dans la composante connexe du sommet i , (il existe un chemin de longueur nulle de i vers i).

On choisit de représenter un tel graphe par la donnée de la liste des voisins de chaque sommet. Ici, les sommets seront simplement des entiers. Un graphe sera informatiquement un objet dont le type est défini par

`type graphe = int list array`: ce sera un tableau `g` de longueur $n = \text{card}(G)$, de sorte que pour tout $i \in \llbracket 0, n-1 \rrbracket$, `g.(i)` est la liste des sommets du graphe. Par exemple, on représente le graphe de la figure qui suit par le tableau:

```
[| [1;2]; [0;5]; [0;3;4;5]; [2]; [2]; [1;2]; [8]; [8]; [6;7] |];;
```



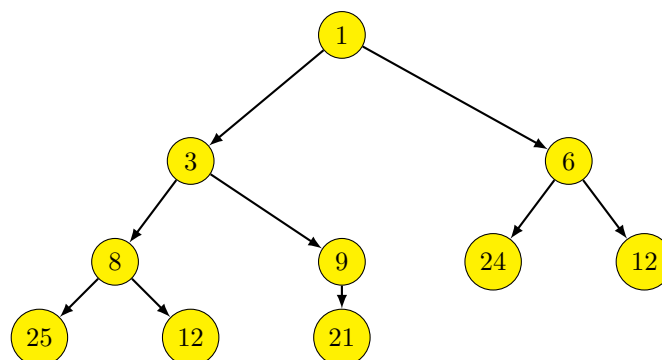
Dans toute la suite, on note G un graphe représenté par l'objet informatique g .

- 1) Écrire une fonction `degre g i` qui renvoie le degré du sommet i de G , c'est-à-dire le nombre de sommets du graphe G qui sont reliés par un chemin de longueur 1 au sommet i .
- 2) Écrire une fonction `deg_max g` qui renvoie le degré maximal d'un sommet de G .
- 3) Écrire une fonction `composante_connexe g s` qui renvoie la liste des points de la composante connexe du sommet s .
- 4) En déduire une fonction `accessible g i j` qui renvoie un booléen de valeur `true` si et seulement si le sommet j est accessible à partir du sommet i .
- 5) Avez-vous écrit une fonction qui parcourt le graphe plutôt en largeur ou plutôt en profondeur? Si vous avez fait l'un faites l'autre.
- 6) (*) Comment parcourir tous les sommets du graphe?
- 7) a) Écrire une fonction `graphe_iter f g i` qui parcourt la composante connexe du sommet i , et applique la fonction de `t` supposée de type `int -> unit` à chacun des sommets de la composante connexe du sommet i .
b) En déduire une fonction `affiche g i` de type `graphe->int->unit` d'affichage des points de la composante connexe du sommet i .

Exercice 4.4

File de priorité implémentée par un TasMin

Dans cette section, on considère des arbres binaires tassés à gauche, c'est-à-dire que chaque noeud peut admettre 0, 1 ou 2 fils. Mais seul le dernier niveau peut-être incomplet. Si le dernier niveau est incomplet et qu'un noeud N de l'avant dernier niveau ne possède pas deux fils, alors tous les noeuds à droite de ce noeud N n'ont aucun fils. En voici, un exemple:



Si de plus, comme sur notre exemple, chaque noeud porte une étiquette inférieure à tous ses descendants, on dira que notre arbre vérifie la propriété de *TasMin*.

On définit ainsi le type `tasmin` par:

`type 'a tasmin = { mutable taille : int; arbre : 'a array };` Le champ `taille` permet de donner le nombre n d'éléments présent dans notre TasMin. Le champ `arbre` est un tableau de taille excédant $n + 1$, de première case insignifiante, qui contient dans les cases d'indice 1 à n , les n éléments du TasMin, placés dans leur ordre de lecture quand on parcourt l'arbre de haut en bas et de gauche à droite. Les cases du champ `arbre` au-delà de la case d'indice $n + 1$ sont insignifiantes.

Dans la suite, quand on parle d'indice, on parle d'indice dans le tableau du champ `arbre`.

Par exemple, le TasMin ci-dessus est représenté informatiquement par le TasMin:

```
{ taille = 10; arbre = [0; 1; 3; 6; 8; 9; 24; 12; 25; 12; 21; 7; 65; 14; 12; 13; -1] }
```

1) Écrire les fonctions suivantes:

- a) `i_pere : int -> int` telle que `i_pere k` renvoie l'indice du père du noeud d'indice k .
- b) `i_fg : int -> int` telle que `i_fg k` renvoie l'indice du fils gauche du noeud d'indice k .
- c) `i_fd : int -> int` telle que `i_fd k` renvoie l'indice du fils droit du noeud d'indice k .
- d) `etiq : int -> 'a tasmin -> 'a` telle que `etiq k t` renvoie l'étiquette du noeud d'indice k dans le TasMin t .
- e) `fg : int -> 'a tasmin -> 'a` telle que `fg k t` renvoie (lorsqu'il existe) la valeur du fils gauche du noeud d'indice k .
- f) `fd : int -> 'a tasmin -> 'a` telle que `fd k t` renvoie (lorsqu'il existe) la valeur du fils droit du noeud d'indice k .
- g) `echange : int -> int -> 'a tasmin -> unit` telle que `echange k kp t` échange les contenus des noeuds d'indice k et kp .
- h) `is_good_father : int -> 'a tasmin -> bool` telle que `is_good_father k t` renvoie `true` si et seulement si le noeud d'indice k est d'étiquette inférieure à celle de ses fils.

Les deux fonctions qui suivent sont à envisager en relation avec les fonctions `insere` et `pop_min`.

i) `remonte : int -> 'a tasmin -> unit` telle que `remonte k t` remonte par échanges successifs avec un père l'élément x d'indice k de t , lorsque t est un arbre binaire tassé à gauche. De façon, à ce que après ces échanges l'élément x soit inférieur à ses fils et supérieur à son père. (On suppose que l'arbre t est presque un tasmin, le seul problème c'est que certes l'élément d'indice k est inférieur à ses fils, mais il est aussi peut-être inférieur à ses pères. Tout le reste de l'arbre respecte la structure de tasmin).

j) `descend : int -> 'a tasmin -> unit` telle que `descend k t` descend par échanges successifs avec un fils l'élément x d'indice k de t , lorsque t est un arbre binaire tassé à gauche. De façon, à ce que après ces échanges l'élément x soit inférieur à ses fils et supérieur à son père. (On suppose que l'arbre t est presque un tasmin, le seul problème c'est que certes l'élément d'indice k est supérieur à ses pères, mais il est aussi peut-être supérieur à ses fils. Tout le reste de l'arbre respecte la structure de tasmin).

k) `insere : 'a -> 'a tasmin -> unit` telle que `insere x t` insère l'élément x dans le TasMin t , de façon à préserver la structure de TasMin

l) `pop_min : 'a tasmin -> 'a` telle que `pop_min t` renvoie l'élément minimal de t et le supprime du tas t , en préservant la structure de tasmin.

m) `liste_to_tas : 'a list -> 'a tasmin` qui transforme une liste en TasMin.

n) `tas_to_list : 'a tasmin -> 'a list` qui transforme un TasMin en une liste triée par ordre croissant.

2) On peut maintenant utiliser la structure de TasMin pour implémenter des files de priorités. On va utiliser un TasMin dans lequel on met des couples de la forme *(priorité, valeur)*.

On définit donc le type file de priorité par: `type 'a fileprio = (int * 'a) tasmin`

(On suppose ici que nos priorités sont des entiers).

Écrire les fonctions suivantes:

- a) `creer_file_vide : 'a -> int -> 'a tasmin` telle que `creer_file_vide x nmax` crée une file de priorité vide dont les éléments sont de même type que x et qui pourra contenir au maximum $nmax$ éléments.
- b) `enfiler : 'a -> 'b -> ('b * 'a) tasmin -> unit` telle que `enfiler elem prio file` ajoute à file l'élément $elem$ affecté de la priorité $prio$.
- c) `popmin_file : 'a fileprio -> int * 'a` telle que `popmin_file file` renvoie le couple *(priorité, valeur)* de la file et le supprime de la file.

Exercice 4.5 Travail sur des graphes non orientés valués: fabrication d'un arbre couvrant minimal

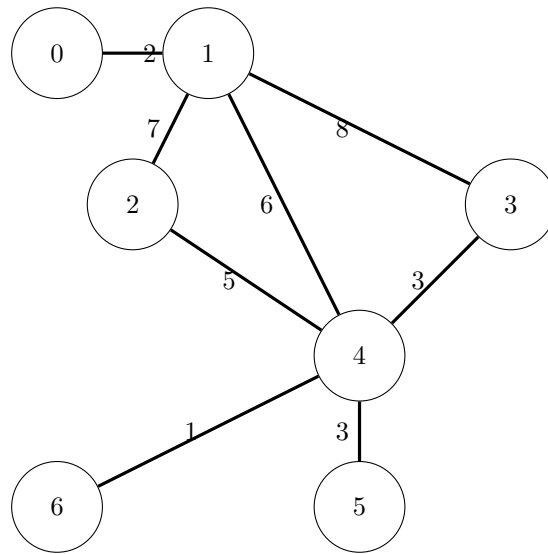
On appelle cette fois-ci graphe un ensemble $G = (S, A)$:

S est l'ensemble des sommets de G , que l'on prendra ici de la forme $\llbracket 0, n - 1 \rrbracket$.

et A est l'ensemble des arêtes de G . C'est un ensemble de triplets de la forme (v, i, j) où i et j sont deux éléments

distincts de S tels que $i < j$ et v est un réel strictement positif. On choisit de représenter un tel graphe par la liste des triplets (v, i, j) de A . Par exemple, le graphe suivant est représenté par la liste:

$[(2; 0; 1); (8; 1; 3); (6; 1; 4); (7; 1; 2); (5; 2; 4); (3; 3; 4); (3; 4; 5); (1; 4; 6)]$



Jules César, arrive en Gaule et souhaite créer des voies romaines pour relier les colonies romaines nouvellement créées. Il a fait étudier par ses architectes le coût des routes directes d'une colonie à une autre. Il souhaite maintenant créer son réseau de façon à dépenser le moins possible. Comment aider Jules? On choisit de représenter les arêtes et les graphes à l'aide des types suivants:

```

type sommet = int
type arete = int * sommet * sommet;;
type graphe = arete list;;

```

1) Travail préparatoire

On veut fabriquer une structure `compo` de données mutable qui permette de représenter les différentes composantes connexes d'un graphe non orienté dont les sommets sont numérotés de 0 à $n - 1$: On veut pouvoir numéroté les différentes composantes connexes et disposer des fonctions suivantes:

- `est_copain comp som1 som2` de type `compo -> sommet -> sommet -> bool` qui renvoie un booléen de valeur `true` si et seulement si les sommets numérotés `som1` et `som2` sont dans la même composante connexe. Les composantes étant représentées par la structure `compo`.
- `fusion comp som1 som2` de type `compo -> sommet -> sommet -> unit` qui fusionne les composantes connexes des sommets `som1` et `som2`.
- `liste_copains som` de type `compo -> sommet -> sommet list` qui renvoie la liste des sommets de la composante connexe du sommet `som`.

a) Que proposez-vous?

b) On va ici simplement coder les composantes connexes par un tableau `t` de n entiers, de sorte que `t.(i)` contienne le minimum des numéros des sommets appartenant à la composante connexe du sommet `i`.

i) Donner les fonctions `est_copain`, `fusion` et `liste_copains`.

ii) Évaluer leur complexité.

iii) En déduire une fonction `composantes g` de type `graphe -> int array` qui pour un graphe donné `g` renvoie le tableau codant les composantes connexes du graphe `g`.

c) Comment donner une structure plus efficace?

2) Résolution: Construction des routes

a) Justifier qu'un réseau routier répondant à la question de Jules est nécessairement un arbre (c'est-à-dire un graphe sans circuit).

b) On va employer l'algorithme de Kruskal suivant:

On construit le graphe R des routes à construire de la manière suivante:

Initialisation: Le graphe R contient tous mes sommets numérotés et aucune arête.

Boucle: Tant que R n'est pas connexe, on ajoute à R la route de coût minimal parmi les routes reliant des sommets n'appartenant pas à la même composante connexe, et l'on fusionne les composantes connexes de ces sommets.

Coder la fonction `kruskal` de type `graphe -> graphe`, qui renvoie les routes à construire pour fabriquer un arbre couvrant minimal du graphe mis en argument.

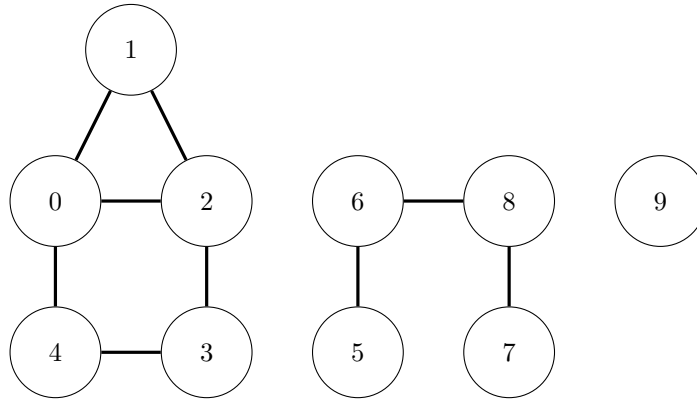
c) Évaluer la complexité de cette fonction, on peut assez facilement obtenir une complexité en $\Theta(n^2 \ln_2(n))$ (avec n égal au nombre de sommets du graphe). Si ce n'est pas le cas, améliorer votre fonction.

Exercice 4.6

Structure union-find

On cherche à représenter les composantes connexes d'un graphe non orienté. Un graphe est un couple (S, A) formé d'un ensemble de S et d'un ensemble d'arêtes A , chaque arête étant une partie $\{i, j\}$ formée de deux éléments de S . On dit que deux sommets s et s' de S sont dans la même composante connexe s'il existe un chemin de s vers s' , c'est-à-dire s'il existe une suite finie de p sommets $s = a_0, a_1, \dots, a_p = s'$ telle que $\forall i \in \llbracket 0, p-1 \rrbracket, \{a_i, a_{i+1}\} \in A$.

Par exemple, on considère le graphe G_1 suivant:



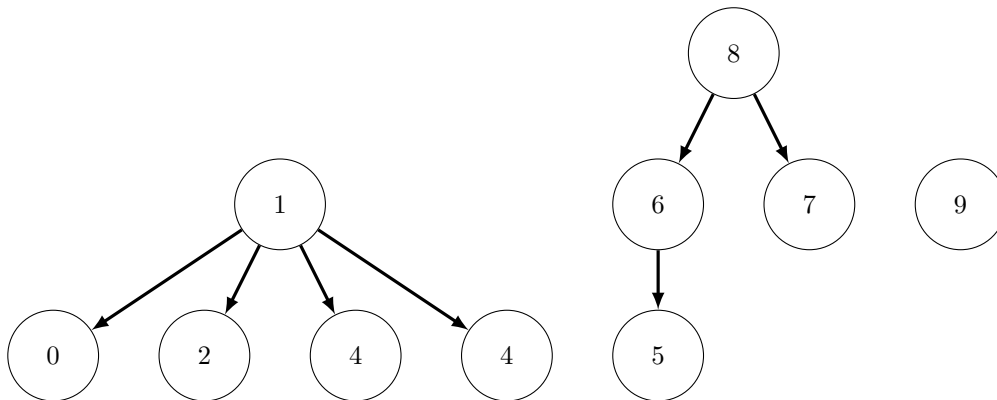
Dans ce graphe, il y a 3 composantes connexes: la première regroupe $\{0, 1, 3, 4\}$, la seconde $\{5, 6, 7, 8\}$ et la dernière est $\{9\}$.

On veut coder une structure qui permette d'effectuer facilement les opérations suivantes:

1. Vérifier si deux sommets sont dans la même composante connexe.
2. réunir deux composantes connexes.

Pour cela, nous allons utiliser une forêt. Chaque composante connexe est représentée par un arbre.

Par exemple, la forêt suivante pourra représenter les composantes connexes du graphe G_1 présenté ci-dessus.



En utilisant deux tableaux

On choisit dans un premier temps de traiter le cas où les sommets sont numérotés de 0 à $n-1$. On représente une forêt à l'aide de deux tableaux.

Le premier est le tableau des pères: `pere.(i)` donne le numéro du père de i . Si i n'a pas de père, alors on décide qu'il est son propre père. Le deuxième tableau est le tableau des hauteurs. Lorsque i est une racine d'un arbre de la forêt, alors `hauteur.(i)` donne la hauteur de l'arbre de racine i . Si i n'est pas une racine, alors `hauteur.(i)` est sans signification particulière.

On utilise donc les type suivants: `type sommet = int;;`

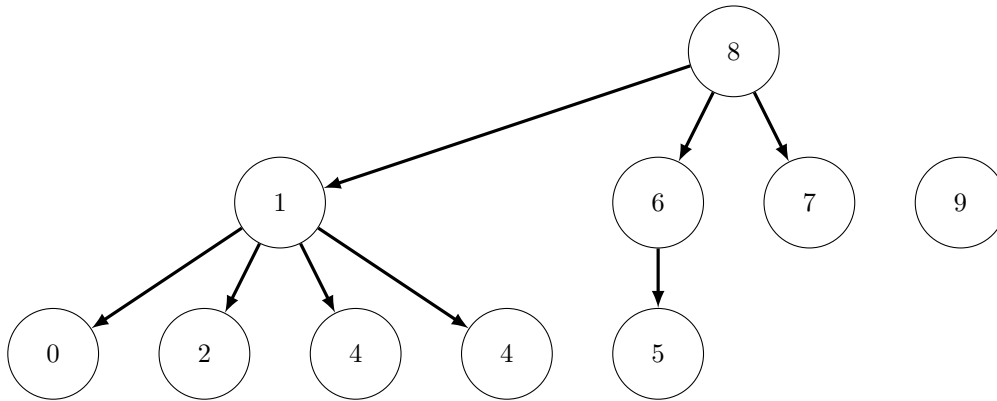
`type unionfind = {pere : sommet array; hauteur: int array};;`

1) Écrire la fonction d'en-tête `chacun_pour_soi (n: int): unionfind` qui renvoie la structure représentant la forêt qui représente les composantes du graphe d'ensemble de sommets $\llbracket 1, n \rrbracket$ et qui ne comporte aucune arête.

2) On suppose que `uf` représente les composantes connexes d'un graphe G . Écrire la fonction d'en-tête `racine (s: sommet) (uf: unionfind): sommet` qui renvoie le sommet racine de l'arbre codant la composante connexe du sommet s .

3) Coder la fonction d'en-tête `est_relie (s: sommet) (sp: sommet) (uf: unionfind): bool` qui permet de savoir si les sommets `s` et `sp` sont dans la même composante connexe.

4) La fonction décisive de la structure **union-find** est la fonction qui permet de fusionner les composantes connexes de deux sommets s et s' . Pour effectuer cette fusion, on récupère les racines r_s et $r_{s'}$ des arbres correspondant à leur composantes connexes respectives. Et on donne, à la racine de l'arbre de plus petite hauteur un nouveau père: la racine de l'arbre de plus grande hauteur. Par exemple, dans la forêt représentée ci-dessus, fusionner les composantes connexes de 2 et 7 reviendra à obtenir la forêt suivante:



Si l'on fusionne deux arbres de même hauteur, on donne arbitrairement à la racine d'un des deux arbres, la racine de l'autre arbre. Écrire la fonction d'en-tête `fusionne (s: sommet) (sp: sommet) (uf: unionfind): unit` qui agit par effet de bord sur `uf` pour fusionner les composantes connexes des sommets `s` et `sp`.

5) On effectue les opérations suivantes:

```

let g1 = chacun_pour_soi 10;;
fusionne 0 1 g1;;
fusionne 1 2 g1;;
fusionne 2 3 g1;;
fusionne 3 4 g1;;
fusionne 5 6 g1;;
fusionne 7 8 g1;;
fusionne 5 7 g1;;
g1;;

```

Vérifier que l'on obtient la forêt représentant G_1 vue ci-dessus. Elle peut être codée par:

```

{pere = [|1; 1; 1; 1; 1; 6; 8; 8; 8; 9|];
 hauteur = [|0; 1; 0; 0; 0; 0; 1; 0; 2; 0|]}

```

6) On agit sur nos composantes connexes à l'aide des fonctions `racine`, `est_relie`, `fusionne`. Vérifier que l'on maintient la propriété suivante: tout arbre de racine s contient au moins $2^{\text{hauteur}(s)}$ sommets. En déduire une majoration de la complexité des fonctions `racine`, `est_relie`, `fusionne`.

En utilisant un type construit pour chaque noeud

On choisit maintenant d'utiliser un nouveau type pour représenter chaque noeuds des arbres représentant une forêt.

On utilise le type défini par:

```

type 'a sommet =
{valeur : 'a; mutable pere : 'a sommet option; mutable hauteur : int};;

```

Le champ `valeur` représente l'étiquette du sommet du graphe, le champ `pere` prend la valeur `None` si le sommet est une racine d'un arbre et prend la valeur du sommet père sinon. Le champ `hauteur` n'a de signification que si le sommet est une racine d'arbre, dans ce cas, il donne la hauteur de l'arbre.

7) Écrire les fonctions suivantes:

a) `make_set : 'a -> 'a sommet` qui transforme une étiquette un objet de type `'a sommet`. (Cette fonction permet de représenter un singleton).

b) `find : 'a sommet -> 'a sommet` qui renvoie le sommet racine du sommet passé en argument.

c) `unis : 'a sommet -> 'a sommet -> bool` qui vérifie si les deux sommets passés en arguments sont ou non dans la même composante connexe.

d) `union : 'a sommet -> 'a sommet -> unit` qui réunit les composantes connexes des deux sommets passés en argument.

Exercice 4.7

Graphes orientés non valués-Tritopologique

On appelle dans cet exercice graphe un couple $G = (S, A)$ de deux ensembles:

S est l'ensemble des sommets de G , que l'on prendra ici de la forme $\llbracket 0, n-1 \rrbracket$

et A est l'ensemble des arcs de G . C'est un ensemble de couples de la forme (i, j) telles que $(i, j) \in S^2$ et $i \neq j$.

Soit $G = (S, A)$ un graphe fini. Un **tri topologique** est une injection f de l'ensemble des sommets vers $\mathbb{R}$, telle que si $a = (i, j) \in A$ est une arête alors $f(i) < f(j)$.

Si i est un sommet du graphe G , on dit que j est un voisin sortant de G si (i, j) est une arête.

On représente un tel graphe par le tableau des listes des voisins sortants de chaque sommet. En caml, on utilisera donc le type suivant: `type graphe = int list array`.

On appelle degré entrant d'un sommet s le nombre noté $d_-(x)$ de sommets $i \in S$ tels que (i, x) est une arête.

1) Écrire une fonction `entrant g` de type `graphe -> int array` qui renvoie le tableau `t` des degrés entrants de chacun des sommets du graphe à n sommets dont `g` est la liste de voisins. C'est-à-dire qu'on a $\forall i \in \{0, \dots, n-1\}$, $d_-(i) = t.(i)$.

2) Montrer qu'un graphe fini sans circuit contient au moins un sommet de degré entrant égal à 0.

3) Montrer qu'un graphe est sans circuit si et seulement si il admet un tri topologique.

4) Écrire un fonction qui détermine si un graphe est ou n'est pas sans circuit et renvoie, le cas échéant, un tri topologique: On écrira une fonction `tritopo` de type `graphe -> int array` de sorte que `tritopo g` renvoie un tableau `f` de première case égale à -1 s'il n'existe pas de tritopologique, et renvoie le tableau des $f(i)$ pour $i \in \llbracket 0, n-1 \rrbracket$ où f est un tritopologique de g , si un tel tritopologique existe.

Exercice 4.8

TimSort

On va mettre en place ici l'algorithme utilisé par Python ou Java pour trier des tableaux d'entiers. L'idée générale est de découper le tableau de longueur n à trier en tranches monotones, puis de fusionner à l'aide d'une fonction analogue à celle du trifusion ces tranches monotones de façon à trier le tableau, la stratégie de choix de l'ordre dans lequel on effectue les fusions est ajustée pour que l'on se retrouve autant que possible à fusionner des tranches de largeur comparable, assurant ainsi au final une exécution en $O(n \ln(n))$.

1) On appelle *run* une tranche monotone (croissante ou décroissante) de largeur maximale, on obtient la décomposition du tableau en *run* de la façon suivante:

(-) Le premier *run* est la plus longue tranche monotone commençant à la première case du tableau.

(-) Si l'on note f_0 l'indice de la dernière case du premier *run*, alors le deuxième *run* est la plus longue tranche monotone commençant à la case $f_0 + 1$.

(-) etc.

Chaque *run* est codé par le couple (d, f) donnant les indices de première et dernière case du *run*.

Écrire la fonction `tab_to_run` de type `'a array -> (int * int) list` qui renvoie la liste des *runs* du tableau passé en argument.

2) Écrire la fonction `merge` de type `int array -> int * int -> int * int -> unit` telle que `merge tab r1 r2` fusionne deux *runs*, supposés croissants, (d_1, f_1) et (d_2, f_2) consécutifs du tableau: elle agit par effet de bord sur `tab` de sorte qu'après exécution la plage (d_1, f_2) de `tab` ne forme plus qu'un seul *run* croissant.

3) Écrire la fonction `merge_brut` qui procède de même mais accepte en argument des *run* croissants ou décroissants.

4) Voici alors l'algorithme du TimSort():

Algorithm 3: TimSort: translation of Algorithm 1 and Algorithm 2

```

Input: A sequence to  $S$  to sort
Result: The sequence  $S$  is sorted into a single run, which remains on the stack.
Note: At any time, we denote the height of the stack  $\mathcal{R}$  by  $h$  and its  $i^{\text{th}}$  top-most run (for  $1 \leq i \leq h$ ) by  $R_i$ . The size of this run is denoted by  $r_i$ .

1 runs  $\leftarrow$  the run decomposition of  $S$ 
2  $\mathcal{R} \leftarrow$  an empty stack
3 while runs  $\neq \emptyset$  do                                     // main loop of TIMSORT
4   remove a run  $r$  from runs and push  $r$  onto  $\mathcal{R}$              // #1
5   while true do
6     if  $h \geq 3$  and  $r_1 > r_3$  then merge the runs  $R_2$  and  $R_3$  // #2
7     else if  $h \geq 2$  and  $r_1 \geq r_2$  then merge the runs  $R_1$  and  $R_2$  // #3
8     else if  $h \geq 3$  and  $r_1 + r_2 \geq r_3$  then merge the runs  $R_1$  and  $R_2$  // #4
9     else if  $h \geq 4$  and  $r_2 + r_3 \geq r_4$  then merge the runs  $R_1$  and  $R_2$  // #5
10    else break
11 while  $h \neq 1$  do merge the runs  $R_1$  and  $R_2$ 

```


En commençant par écrire les fonctions d'en-tête `main.loop runs pile tab` qui réalise la boucle principale et `fusion_finale tab pile` qui effectue les fusions finales, écrire le code de la fonction `timsort` de type `'a array -> unit` qui trie le tableau passé en argument.

5) [Chaud patate] Prouver la validité et la complexité du TimSort.

Exercice 4.9

Dictionnaires

Un dictionnaire est une structure de donnée où l'on stocke des éléments dotés d'une clé, de sorte que l'on puisse extraire ou lire ou ajouter un élément correspondant à une clé donnée.

Exemples de dictionnaires:

Un Robert est un dictionnaire dont les clés sont les mots de la langue française, les éléments ou valeurs associées à ces clés sont les définitions de ces mots.

La liste des contacts enregistrés dans un téléphone est un dictionnaire dont les clés sont les noms des personnes et les valeurs associées à ces clés sont les numéros de téléphone de ces personnes.

On définit le type dictionnaire à partir de deux types, le type `'a` utilisé pour coder les clés, le type `'b` utilisé pour coder les valeurs. On définit alors le type `('a, 'b) dictionnaire` dont les fonctions (pour une implémentation impérative, avec des structures mutables:) sont:

- `dicoVide` de type `'a -> 'b -> ('a, 'b) dictionnaire` qui crée un dictionnaire vide dont les clés seront de type `'a` et les valeurs de type `'b`.
- `ajouter` de type `('a, 'b) dictionnaire -> 'a -> 'b -> unit` telle que `ajouter dico cle val` ajoute le couple `cle, val` au dictionnaire s'il n'y avait pas d'élément de clé `cle` et remplace l'ancienne valeur pour la clé `cle` par la valeur `val` sinon. (Pour la version sans doublon: ou une clé est associée à une seule valeur, mais on peut aussi envisager une version avec des doublons possibles, par exemple pour des mots polysémiques dans le dictionnaire de la langue française).
- `defini` de type `('a, 'b) dictionnaire -> 'a -> bool` qui permet de tester s'il existe un élément de clé donnée dans le dictionnaire.
- `association` de type `('a, 'b) dictionnaire -> 'a -> 'b` qui permet de renvoyer la valeur (ou les valeurs si on s'autorise les doublons) associée à une clé donnée dans un dictionnaire.
- `estVide` de type `('a, 'b) dictionnaire -> bool` qui teste si un dictionnaire est vide ou non.
- `enlever` de type `('a, 'b) dictionnaire -> 'a -> unit` qui permet d'enlever la valeur associée à une clé donnée.

On dispose aussi souvent de la fonction `dico_to_liste` de type `('a, 'b) dictionnaire -> ('a * 'b) list` qui convertit le dictionnaire en une liste de couples.

1) Quel seraient le type de ces fonctions si l'on employait une structure persistante?

A. Implémentations naïves

2) Implémentation par structure persistante naïve

On choisit d'implémenter le type `('a, 'b) dictionnaire` par des listes de couples de type `('a * 'b) list`. Écrire les fonctions usuelles pour cette implémentation. On adaptera le typage des fonctions précédentes: on utilisera ici une structure non mutable de dictionnaire et par exemple, la fonction `ajouter` renverra un dictionnaire. Discuter de la complexité de ces fonctions.

3) Implémentation par structure mutable naïve

a) On suppose que les clés sont des entiers de l'intervalle $\llbracket 0, N_{max} - 1 \rrbracket$, on peut alors utiliser comme dictionnaire des tableaux de `Nmax` valeurs. Écrire les fonctions usuelles pour cette implémentation. Discuter de la complexité de ces fonctions.

b) On veut adapter cette représentation au dictionnaire des contacts d'un téléphone. On se donne donc une bijection f de l'ensemble des mots de moins de 10 lettres vers un ensemble d'entiers. On se ramène au cas précédent. Proposez un exemple de bijection possible. Discuter de l'intérêt de cette méthode.

B. Table de hachage

La méthode précédente nécessite donc de disposer d'une fonction injective de l'ensemble (souvent immense) des clés possibles vers l'ensemble des entiers de 0 à $N_{max} - 1$. L'injectivité impose donc des valeurs de N_{max} immenses. On va donc faire une croix sur l'injectivité et utiliser des fonctions non injectives de l'ensemble des clés possibles vers un ensemble de type $\llbracket 0, N_{max} - 1 \rrbracket$ pour une valeur de N_{max} raisonnable. Une telle fonction est appelée **fonction de hachage**. Évidemment apparaissent alors des conflits lorsque deux clés différentes ont la même image par la fonction de hachage, on appelle ce conflit une **collision**.

Pour résoudre ces conflits, on peut, par exemple:

- Effectuer nos insertions en sondant les cases à côté de la case prévue initialement jusqu'à trouver une case vide. L'ordre des cases à sonder devant être déterminé par une fonction de hachage préalablement améliorée. (On parle d'adressage ouvert).
- Placer dans une même liste toutes les valeurs dont les clés ont même image par la fonction de hachage.

C. Un exemple d'implémentation par chaînage

On définit le type dictionnaire par

```
type ('a,'b) dictionnaire = {h: 'a -> int; contenu: ('a* 'b) list array};;
```

Le premier champ d'un dictionnaire sera donc une fonction de hachage, et le second la liste des couples *cle, valeur* du dictionnaire, si $h(cle) = i$, alors le couple $(cle, valeur)$ sera mis dans la liste contenue dans la case d'indice i du champ *contenu* du dictionnaire.

4) Un exemple de fonction de hachage sur les mots. Pour un mot $w = l_0 l_1 \dots l_p$ de l'alphabet usuel, on pose $f(w) = \sum_{i=0}^{p-1} c(l_i) 27^i$ où $c(a) = 1$, $c(b) = 2$, et *caetera*. (On a $c(z) = 27$). si le mot w n'a pas de i -ème lettre $c(i) = 0$. Et on note $exh1(w)$ le reste dans la division de $f(w)$ par 30. On pourra, ensuite, implémenter un tout petit dictionnaire contenant, par exemple, les définitions suivantes:

- "bonjour", "formule de politesse du matin"
- "rame", "outil pour la galère"
- "came" , "outil pour ne pas ressentir qu'on galère"
- "ves", "cales en forme de lettres V,"
- "ave maria", "priere a la sainte vierge"
- "ria", "vallee fluviale envahie par la mer"

Écrire la fonction `dicoVide f nMax` qui renvoie un dictionnaire vide dont la fonction de hachage est f lorsque l'on suppose que f est à valeurs dans $\llbracket 0, NMax-1 \rrbracket$.

5) Écrire la fonction d'ajout telle que `ajouter dico cle valeur` ajoute le couple `cle, valeur` au dictionnaire `dico`.

6) Créer pour les tests suivants le dictionnaire `dico1` dans le quel vous mettrez les exemples ci-dessus et quelques exemples de votre choix.

7) Écrire de même les fonctions `defini dico cle, association dico cle, estVide, enlever dico_vers_liste`.

8) Tester ces fonctions et discuter de leur complexité.

D. Le module `hashtbl` de OCAML

```
(* creation d'une table de taille estimee 123456*)
let my_hash = Hashtbl.create 123456;;
(*ajout d'un element a une table**)
Hashtbl.add my_hash cle valeur;
(*renvoi du dernier element mis dans la table de cle donnee*)
Hashtbl.find my_hash cle;;
(*renvoi tous les elements de la table de cle donnee*)
Hashtbl.find_all my_hash cle;;
(*suppression d'un element*)
Hashtbl.remove my_hash cle;;
(* existence d'un element de cle donnee*)
Hashtbl.mem my_hash cle
```

E. Exercice classique

On se donne un tableau ou une liste t d'entiers naturels deux à deux différents et s un entier naturel. On cherche à renvoyer la liste de tous les couples (i, j) tels que $t.(i) + t.(j) = s$.

9) Écrire une fonction naïve pour répondre au problème proposé. Traiter les deux cas: l'argument est une liste et l'argument est un tableau. Complexité?

10) Écrire une fonction qui trie la liste en entrée et la met dans un tableau puis répond au problème. Complexité?

11) Écrire une fonction utilisant les tables de hachage de OCAML. On supposera que l'on place une liste en entrée.

12) Écrire une fonction utilisant une table de hachage "fait-maison". On traitera les deux cas: les données en entrée sont dans une liste ou dans un tableau. Complexité?

Exercice 4.10

Graphes eulériens

On considère des graphes orientés sans boucle de la forme (S, V) où S l'ensemble des sommets est un ensemble de la forme $\llbracket 0, n-1 \rrbracket$ pour $n \in \mathbb{N}^*$ et V l'ensemble des arcs est un ensemble de couples (v, v') pour $v \in S$ et $v' \in S$ tels que $v \neq v'$.

Un *chemin* d'origine $u \in V$ et d'extrémité $v \in V$ est défini par une suite d'arcs consécutifs reliant u à v .

Un *circuit* est un chemin dont l'origine et l'extrémité sont le même sommet et dont les arcs deux à deux distincts. (Il peut néanmoins passer plusieurs fois par le même sommet). (Un circuit peut être réduit à un point).

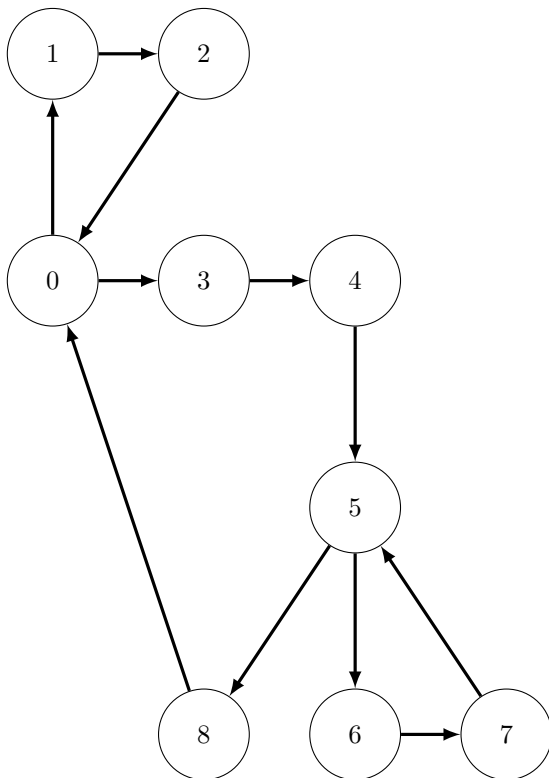
Un chemin (resp. circuit) est dit *eulérien* s'il contient une fois et une seule chaque arc de G .

Soit $(u, v) \in E$. On dit que l'arc (u, v) est un *arc entrant* du sommet v et un *arc sortant* du sommet u . On définit le *degré entrant* d'un sommet v , noté $d_e(v)$, comme le nombre d'arcs entrants du sommet v . On définit le *degré sortant* d'un sommet u , noté $d_s(u)$, comme le nombre d'arcs sortants du sommet u .

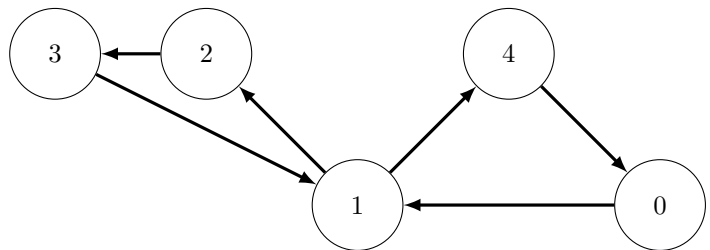
On dit que v est un *voisin* de u dans le graphe G si (u, v) est un arc de G .

On dira qu'un graphe est *équilibré* si $\forall v \in V, d_e(v) = d_s(v)$.

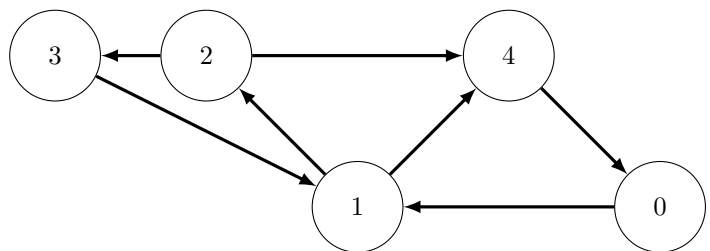
Le graphe G_1 admet un circuit eulérien.



Le graphe G_2 admet un circuit eulérien.



Le graphe G_3 n'admet pas de circuit eulérien, mais il admet un chemin eulérien.



1) Montrer que si un graphe orienté sans point isolé contient un circuit eulérien alors le graphe est équilibré et fortement connexe.

2) a) Soit G un graphe équilibré et un sommet de ce graphe tel que $d_s(v) > 0$. Justifier qu'il existe un circuit (pas forcément eulérien) qui commence en v et finit en v .

b) On se donne un graphe G connexe et équilibré. Justifier que si C est un circuit du graphe G qui n'est pas eulérien, alors il existe au moins un sommet du circuit qui est l'origine d'un arc qui ne fait pas partie du circuit.

c) Prouver que tout circuit non eulérien d'un graphe connexe équilibré peut être allongé un circuit eulérien strictement plus grand.

d) Prouver que tout graphe connexe équilibré admet un circuit eulérien.

3) On représente un graphe d'ensemble de sommets $\llbracket 0, n-1 \rrbracket$ par le tableau $[\ell_0, \dots, \ell_{n-1}]$ où pour tout $i \in \llbracket 0, n-1 \rrbracket$, la liste ℓ_i est la liste des voisins sortant du sommet i .

On définit les types:

```
type sommet = int;;
type arc = int*int;;
type graphe = {n:int; liste_adj : int list array};;
```

Par exemple, le graphe G_1 est représenté par:

```
let ex1 = {n = 9; liste_adj = [| (*v0*) [1;3]; (*v1*) [2]; (*v2*) [0];
(*v3*) [4]; (*v4*) [5]; (*v5*) [6;8]; (*v6*) [7];
```

```
(*v7*)[5]; (*v8*)[0]; []];;
```

Le graphe G_2 est représenté par:

```
let ex2 = {n = 5; liste_adj = [| [1]; [2;4]; [3];[1]; [0] |]};;
```

a) Définir la fonction `est_connexe : graphe -> bool` telle que `est_connexe g` renvoie `true` si et seulement si `g` est connexe.

b) Définir la fonction `est_equilibre : graphe -> bool` telle que `est_equilibre g` renvoie `true` si et seulement si `g` est équilibré.

c) On définit donc:

```
let admet_circuit_eulerien g = (est_equilibre g) && (est_connexe g);;
```

qui permet de savoir si un graphe admet un circuit eulérien.

d) Écrire la fonction `fait_bb_circuit : sommet -> graphe -> sommet list` de sorte que `fait_bb_circuit v g` renvoie la liste des sommets successifs d'un circuit du graphe représenté par `g`. (On suppose que $d_s(v) > 0$ et que G est équilibré). La fonction agira de plus par effet de bord sur G pour en supprimer tous les arcs empruntés.

e) Écrire une fonction `circuit_g : sommet -> sommet list` qui renvoie la liste des sommets successifs d'un circuit eulérien du graphe (supposé connexe et équilibré) passé en argument.

4) Comment traiter le cas de la recherche de chemins eulériens.